GRIDDO PATH-FINDING ANALYSIS

[redacted]

MAY 26, 2025

1 Introduction

This project aims to efficiently compute the fastest paths between a character and a player-determined destination in the computer game **Griddo**.

2 Game Features

Some salient features of the game include the map, terrain, and rain.

In terms of the map, the game operates on a rectangular unit grid where characters are initially placed on a square. Players can move in 4 cardinal directions to adjacent squares (no diagonal movement).

Additionally, each square is assigned a permanent terrain type from one of:

- **Field**: Field squares allow for normal movement speed meaning they can be covered in a single unit of time (this makes the idea of a 'fast' path more concrete).
- **Forest**: Forest squares half movement speed, consequently doubling the time required to cover them. Thus, they can be crossed in 2 units of time.
- **Mountain**: Mountain squares cannot be passed through, and thus for modelling purposes, the time taken to cover them is considered to be infinite.

Finally, we assume characters maintain a uniform movement speed across a particular tile (the character does not speed up or slow down over a square).

2.1 Rain

Rain dynamically affects squares throughout the game, halving speed across the square, and doubling its original movement time. For this report we assume that rain is implemented probabilistically, and the squares are randomly modified after each character move. This is slightly unrealistic as in the real world rain is not constrained to boxes, and is dependent on time rather than when a move occurs. Despite this, these adjustments prevent rain from occupying fractions of a square, or the map changing mid-calculation. Within our game, once the character makes their move, the map is probabilistically updated with rain (including the character's square), changing movement times.

2.2 Rain Specifications

A simple yet unrealistic implementation of rain would be to assign a probability p that every move a square is filled with rain, regardless of its state. Although this is simple to simulate, this type of pure randomness makes it impossible for the agent (character) to predict future states or exploit patterns. This is because the configuration of rain changes after every move, and thus the character can only understand the states of adjacent squares, as the rest are probabilistically equivalent (equal likelyhood of being rainy when the character lands on it). Furthermore, as having rain on adjacent squares cannot be predicted or avoided, the agent is incentivised to stick to the pre-computed shortest path, voiding the point of rain entirely.

Another approach prioritising realism, would look similar to a probabilistic version of Conway's Game of Life, where nodes are influenced by the states of their neighbouring squares, or even the rest of the map. While this approach is more natural, it suffers from too much complexity, where the rain once again becomes random for almost all practical purposes, making it difficult to determine an optimal policy.

Thus, we choose a middle-ground approach, modelling each square as a discrete-time Markov chain with states rainy and dry, updating squares after every move. For the examples, we have used the probabilities $\Pr(\text{Rainy} \to \text{Rainy}) = 0.7$ and $\Pr(\text{Dry} \to \text{Dry}) = 0.9$. This results in approximately 25% of the map covered in rain in the long-run (steady-state), although other probabilities can be equally used for different dynamics. This Markov process can be represented using the transition matrix (row stochastic),

$$P = \begin{array}{c} \\ \text{Rain} \\ \text{Dry} \end{array} \begin{array}{c} \begin{array}{cc} \text{Rain} & \text{Dry} \end{array} \\ \begin{bmatrix} 0.7 & 0.3 \\ 0.1 & 0.9 \end{bmatrix} \end{array}$$

The benefit of this approach is that it is not too complex - rain only depends on the current state of each square. Despite this, it is also not too simple - there is a higher likelihood that dry states will remain dry, and rainy states will remain rainy, incentivising the character to avoid patches of rain.

3 Modelling

We model the map using a Graph ADT, specifically, a weighted undirected graph $G = (V, E)$:

- Each node $v \in V$ corresponds to a square on the grid.
- Each edge $e \in E$ represents valid movement between two adjacent grid squares.

3.1 Node ADTs

The Node ADTs will have an attribute 'Neighbours' that returns the set of all nodes connected to it via a single edge. The Nodes will also have a string ADT to store its terrain type ("field", "forest", or "mountain"), and a boolean attribute for rain status (true if rain, False if no rain).

To extend the model, we could incorporate float ADTs that store the cost of traversing the node as defined below. Despite this, these costs felt more like an imposed algorithmic construct than an intrinsic feature of the game. Thus we included the functionality as a separate algorithm.

3.2 Edge ADTs

Within the graph, edges represent possible moves between adjacent squares. As movement can go in both directions (the character can shift back and forth on two squares), the edges are undirected.

Additionally, to model the varying movement speeds of different terrains, the edges will also maintain a numerical attribute for the time taken to make a particular move. The time taken to cross a field square without rain is defined to be 1, and all other times will be measured relative to that. Finally, these values will be stored as edge weights.

We may refer to the time taken as the cost of a move/path throughout this report.

3.3 Edge Weight Logic

To compute the edge weights, we assume that the character begins and ends their movement in the center of a square. We also use the principle that the net travel time between two squares is the sum of the time to exit one and enter the other.

$$\text{Weight}(u, v) = \text{cost}(u \to \text{center}) + \text{cost}(\text{center} \to v)$$

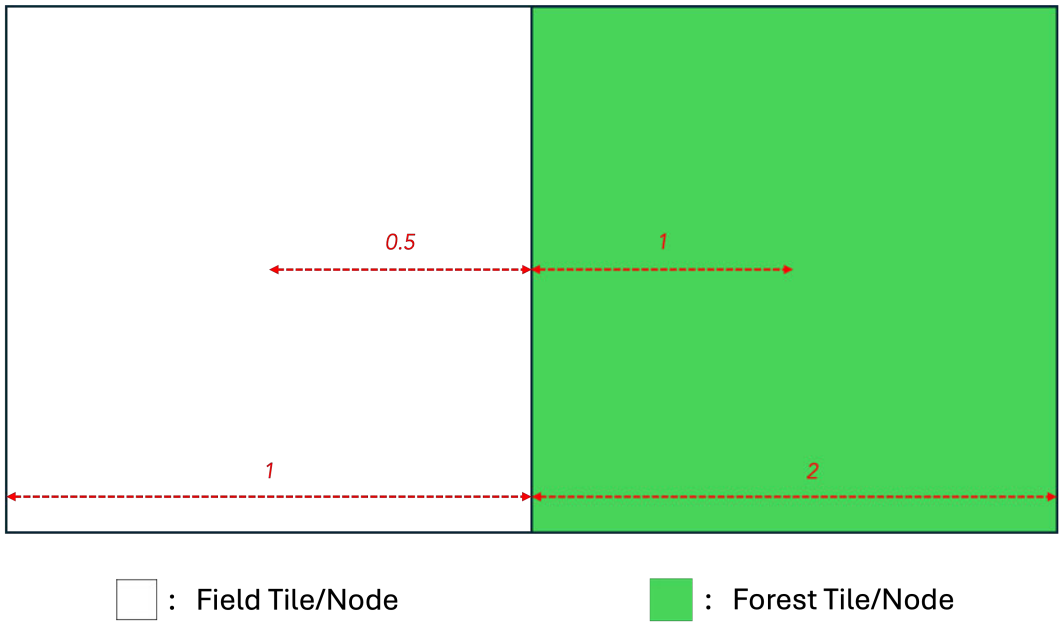


Figure 1: Path cost calculation example.

From the diagram we see that the cost to exit the field square is half of its full traversal cost

($\frac{1}{2} = 0.5$), and the cost to reach the center of the forest square is $2/2 = 1$. This gives a total edge weight of 1.5.

We can generalize this logic to squares containing rain, simply doubling the required values.

If the character follows a path that begins and ends on a square of the same terrain (e.g. forest to forest, or field to field), this approach gives the same path cost as assigning weights to the edges leading in/out of a square. Despite this, for paths that begin and end on different terrains, this method accounts for fractional times and thus computes the true cost.

Finally, we set the edge weights of edges leading into (and consequently out of) mountain squares to infinity. This simulates the property that they are impassable and will deter shortest-path algorithms from exploring them.

It should be noted that an alternative to storing the mountain nodes with infinite weights is to simply remove them, as the squares can technically not be accessed. Despite this, we choose to go with our original approach due to simplicity in understanding the model and ease of assigning coordinates within a complete grid.

Figure 2 shows a representation of a sample map using the proposed ADTs.

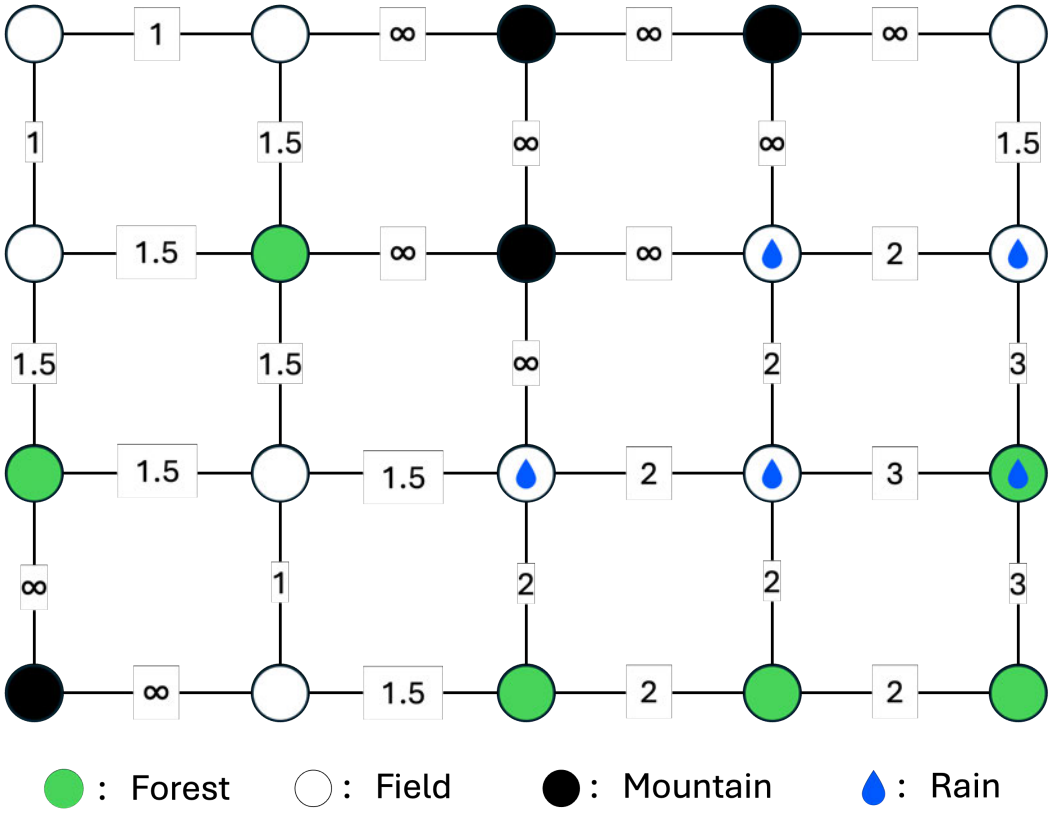


Figure 2: Example map using graph ADTs.

4 Problem Specification

Using our representation of the task, it can now be formulated as a graph traversal problem. In particular, the goal can now be more formally stated as a Single Source Shortest Path Problem with obstacles and dynamic edge weights.

5 ADT Signatures

Signatures ADTs
Neighbours: node → set
IsRain: node → bool
TerrainType: node → string
Steps: node → int
Dist: node → float
Predecessor: node → node
MatrixMult: array × array → array
Power: array × int → array
TerrainCost: node → float
ExpectedCost: node × node × array → float
Trace: node × node × graph → node
ExpectedDijkstra: node × node × graph × array → node
MoveCharacter: node × node × graph × array → string

6 Algorithm Design

From the specification, this is a graph traversal problem with dynamic, probabilistically weighted edges. Thus, a globally optimal solution would be a minimum-cost path that accounts for the exact future weight fluctuations that occur within a particular trial (perhaps dodging rain before it even appears). However, computing such a solution is infeasible, as these weight variations cannot be known with certainty in advance (rain is not deterministic), and the character's moves cannot be returned.

Given that no algorithm can reliably find a globally optimal solution in every map, we define an optimal algorithm to be one that makes the 'best' decision with the information known.

6.1 'Best' Move

Fortunately, due to our model of rain as a Markov process, the relevant information that we must keep track of is just the current board state at any time-step (defining property of Markov processes). Even with this information, it is not directly evident what constitutes the 'best' move. This is illustrated by the following example:

Consider the map in Figure 3 where the goal is to move from the bottom left corner $(0, 0)$, to

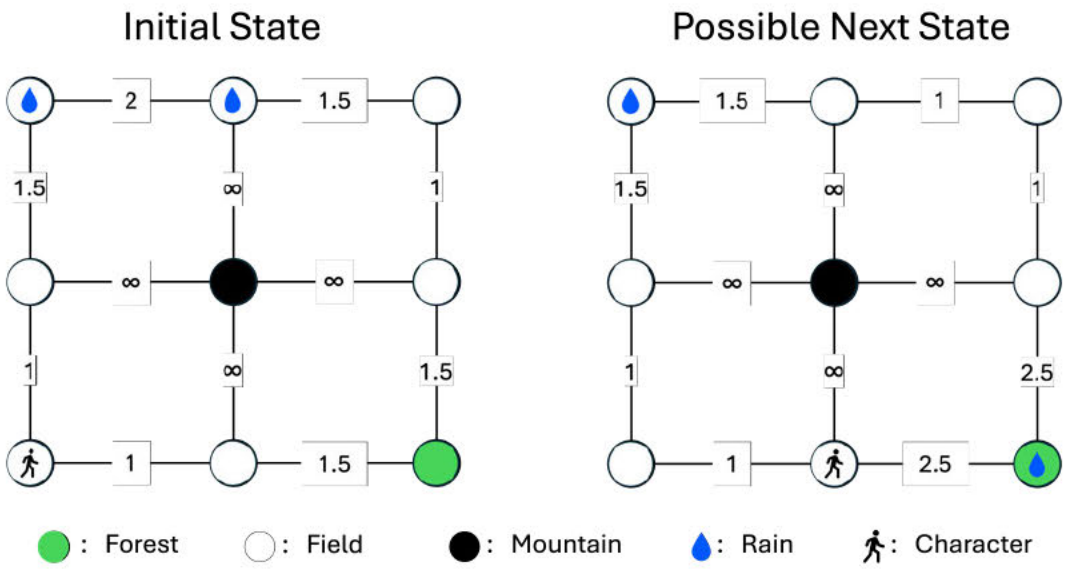


Figure 3: Right: Initial board state. Left: Possible next board state after move right.

the top right corner $(2, 2)$. Applying a shortest path algorithm such as Dijkstra's Algorithm (which is greedy), we see the shortest path within this current board state is $(0, 0) \to (1, 0) \to (2, 0) \to (2, 1) \to (2, 2)$ with a total time of 5. Thus, we may be motivated to state that the best move is to move right.

However, this calculation does not account for the possibility of the board changing and the weighting of different terrain types. In fact, using the earlier defined probabilities, we see that there is only around a 50% probability that square $(1, 2)$ will still be rainy when we enter it and around a 40% chance it will be rainy when we exit. In comparison, there is around a 10% chance that square $(2, 0)$ will be rainy when it is entered and around a 16% it will be rainy when it is exited. As it is a forest square, this would be majorly detrimental to the path's cost.

> **Note:** Due to the midpoint weighting system, half of the square's total cost is incurred when entering, and half of the cost is incurred when exiting.

Therefore, it can be seen that simple path finding does not accurately evaluate the true time/-cost of paths, and consequently moves.

A better metric to consider is the **expected cost** of a path, given the current board state. This accounts for changes in weights by finding the average cost of a collection of moves. As the true path costs are centered around the mean, this is more likely to yield accurate results. The 'best' path can now be seen as one that minimises the expected cost of travelling from the start square to the goal, and the 'best' move is just the first move along the minimising path.

7 Expected Cost Calculation

Consider a path consisting of n edges. Let $X_0, X_1, \ldots, X_{n-1}$ be random variables representing the weight (i.e., time cost) of traversing the ith edge, immediately after the ith move. As the rain process is independent between time steps, so are the random variables. Therefore,

$$\mathbb{E}[\text{Cost} \mid \text{Initial Conditions}] = \mathbb{E}\left[\sum_{i=0}^{n-1} X_i \,\middle|\, \mathcal{H}\right] = \sum_{i=0}^{n-1} \mathbb{E}[X_i \mid \mathcal{H}]$$

Where $\mathcal{H}$ is a random variable representing the intitial board state.

Due to the mid-point weighting model, each edge X_i depends on the terrain and rain status of its two adjacent squares. Thus, we introduce random variables $Z_{k,j}$ which reveal information on the rain and terrain of squares. Mathematically, they denote the time taken to fully traverse across square k after j moves.

The possible values of $Z_{k,j}$ are:

Terrain Type	Dry	Rainy
Field	1	2
Forest	2	4
Mountain	∞	∞

Thus, it can be seen that each edge weight is just the average of the time to traverse its two adjacent squares (half of the time taken to cross the first square, and half of the time taken to cross the second). In other words:

$$X_i = \frac{1}{2}\left(Z_{i,i} + Z_{i+1,i}\right)$$

Since the rain states of individual squares evolve independently as a Markov process, and terrain type is fixed, we take expectations on both sides:

$$\mathbb{E}[X_i \mid \mathcal{H}] = \frac{1}{2}\left(\mathbb{E}[Z_{i,i} \mid \mathcal{H}] + \mathbb{E}[Z_{i+1,i} \mid \mathcal{H}]\right)$$

To compute $\mathbb{E}[Z_{i,j} \mid \mathcal{H}]$, note that the rain status evolves as a Markov process with transition matrix P, and the probability that a square is rainy or dry after j moves depends on P^j and its initial state (rainy or dry). The cost depends on its terrain.

We show the specific calculation of $\mathbb{E}[Z_{i,j} \mid \mathcal{H}]$ by way of an example.

Example: Suppose square 5 along a path is a forest square, initially dry. Using a rain transition matrix P the expected traversal time to cross 5 after 2 moves is:

$$\mathbb{E}[Z_{5,2} \mid \mathcal{H}] = 4 \cdot \mathbb{P}(\text{rainy at } t = 2 \mid \text{dry}) + 2 \cdot \mathbb{P}(\text{dry at } t = 2 \mid \text{dry})$$

where these probabilities are the matrix elements, $p_{2,1}$ and $p_{2,2}$ of P^2. Using a similar logic for other values, we can calculate the expected cost of a path.

7.1 Matrix Representation

A neater way to express the expected values is in terms of matrices. Let the initial rain state determine the basis vector $\mathbf{e} \in \{[1,0],[0,1]\}$, where $\mathbf{e} = [1,0]$ if rainy initially, or $[0,1]$ if dry. This is equivalent to an initial state vector where we are certain of whether the square starts as rainy or dry. Then:

$$\mathbb{E}[Z_{i,j}] = (\mathbf{e}P^j) \cdot \begin{bmatrix} 2a \\ a \end{bmatrix}$$

where the dot denotes the Euclidean dot product, and a controls the cost of the square. It can be either 1, 2 or ∞ depending on the terrain of the square.

Substituting into the expression for $\mathbb{E}[X_i]$ we see:

$$\mathbb{E}[X_i \mid \mathcal{H}] = \frac{1}{2}\left(\mathbf{e}_1 P^i \cdot \begin{bmatrix} 2a \\ a \end{bmatrix} + \mathbf{e}_2 P^i \cdot \begin{bmatrix} 2b \\ b \end{bmatrix}\right)$$

where:

- $\mathbf{e}_1$: initial rain state of square i
- $\mathbf{e}_2$: initial rain state of square $i+1$
- a: field/forest cost coefficient for square i
- b: field/forest cost coefficient for square $i+1$

Finally, we define:

$$\mathbf{v}_i = a\mathbf{e}_1 + b\mathbf{e}_2$$

Which gives,

$$\mathbb{E}[X_i \mid \mathcal{H}] = \mathbf{v}_i P^i \cdot \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

We can now sum over all squares within a path to determine a formula for the expected cost. For simplicity we define $\gamma = [1, 0.5]^T$. Additionally, it can be seen that if any square along the path is a mountain, then $\mathbb{E}[\text{Cost} \mid \mathcal{H}] = \infty$.

$$\mathbb{E}[\text{Cost} \mid \mathcal{H}] = \sum_{i=0}^{n-1} \mathbf{v}_i P^i \cdot \gamma$$

Applying this formula to the map in Figure 3, we see the expected cost of path $(0,0) \to (1,0) \to (2,0) \to (2,1) \to (2,2)$ is 5.586, while the expected cost of path $(0,0) \to (0,1) \to (0,2) \to (1,2) \to (2,2)$ is 5.224. Thus, despite the first path having a shorter length with the initial board state, the expected length correctly identifies the second path as optimal.

8 Algorithm Design

This section outlines our algorithm for computing the optimal path to guide the character. The aim is to find the minimum expected cost path from the character's current location to the goal, given a Markov model governing rain transitions.

Our algorithm is a modified version of Dijkstra's algorithm that takes as input the character's initial position, goal square, and probability matrix (for the rain transition model). The character's starting position and final square will be represented as nodes, and the matrix will be represented via a 2D array. The algorithm outputs the next node along the path with the lowest expected traversal cost. After the character moves and the map updates, the algorithm is rerun to determine the next optimal step. This iterative process continually guides the character along a path that minimizes the net cost of reaching the goal on average.

8.1 Dijkstra's Algorithm

Dijkstra's algorithm is a classical graph traversal method used to compute the shortest paths from a single source node to all other nodes in a graph with non-negative edge weights. The algorithm begins by initializing the distance to every node as infinity, except for the source node, which is assigned a distance of zero. A priority queue (typically a min-heap) is used to repeatedly select the node with the smallest known distance, initially containing only the source node.

At each step, the algorithm extracts the node v' with the minimum tentative distance from the priority queue. It then performs a process called *relaxation* on all neighbors v of v', updating their distances if a shorter path is found. The update rule is given by:

$$d(v) = \min\left(d(v),\ d(v') + w(v', v)\right),$$

where $d(\cdot)$ denotes the current shortest known distance from the source, and $w(v', v)$ is the weight of the edge between v' and v.

Once a node is removed from the priority queue, its shortest path estimate is considered *final*, as it is guaranteed to be optimal due to the non-negative weights and the greedy nature of the algorithm. No shorter path can subsequently be found to this node through unvisited vertices as it already has the lowest distance estimate among them.

This process continues until all reachable nodes have been visited, or a specific target node has been reached. Notably, Dijkstra's Algorithm builds shortest paths to all intermediate nodes in the process of reaching the goal.

8.2 Griddo Algorithm

To calculate the path with the least expected cost, we employ a similar approach to Dijkstra's Algorithm, using *expected* edge weights ($\mathbb{E}[X_i \mid \mathcal{H}]$) instead of the fixed ones.

We begin by initialising the tentative expected time (Dist) of all nodes to infinity, except for the character's start position which is set to 0. We also define a property Predecessor for each node, initialised to null. This allows the character to determine their move at the end of the algorithm Additionally, the expected cost of traversing an edge also depends on the time it is traversed (after how many moves). For example, if a square is initially rainy, it has less chance of still being rainy after 10 moves/steps than it does after 5. To account for this, we initialise a

time attribute (Steps) of the source square to 0, and the other squares to null. Finally, we also maintain a minimum ranking priority queue that takes in a node and the current best estimate for its least expected value. It is initialised to just contain the start node with a priority of 0.

We choose not to include a visited attribute for nodes removed from the priority queue as it would complicate the pseudocode and would not functionally impact the algorithm execution. In our implementation, a node's expected distance is only updated if a shorter path is found. Once a node is removed from the priority queue, it holds the shortest possible distance from the source. Any remaining entries for that node in the queue must represent longer paths, since the shortest one has already been processed. These outdated entries cannot lead to shorter expected paths for neighboring nodes, as their estimated distances would be higher than those already stored. As a result, no updates would occur, and the algorithm's behavior remains correct without explicitly marking nodes as visited.

Similar to Dijkstra's, we extract the node with the least expected value from the priority queue and relax its neighbours. As we are working with expected values, the update rule looks slightly different, although it is fundamentally the same:

$$\mathbb{E}[u \mid \mathcal{H}] = \min\left(\mathbb{E}[u \mid \mathcal{H}], \mathbb{E}[u' \mid \mathcal{H}] + \mathbf{v}_n P^n \cdot \gamma\right)$$

Here, u' is the node extracted from the priority queue, and u is a neighbour of u'. n is the time attribute of u', and $\mathbb{E}[\cdot \mid \mathcal{H}]$ refers to the shortest *known* expected distance from the source. The other variables are defined as earlier. If the latter value is chosen as the minimum, it means we have found a new shortest expected path to node u, and thus we adjust its time attribute to $n+1$ (it is an additional step from u') and change its predecessor value to u'. We then enqueue u' to the priority queue, with its expected value.

Once a node is removed from the priority queue, same as Dijkstra's, its expected path estimate is considered *final* and it is not adjusted again. This process continues until the target node has been reached, or all nodes have been visited in which case the final distance would be infinite (the algorithm went through a mountain). From the algorithm, we can determine the next move by tracing the predecessor chain backward from the goal node. Since we only need the immediate next step (the full path will be recalculated), we start at the goal and follow each node's predecessor until we reach the initial node. The node whose predecessor is the initial node is the next square to move to.

Since expected values are real-valued and non-negative, and future expected edge weights are computed using only the current board state which remains fixed during path computation, the optimal substructure and greedy conditions of Dijkstra still apply. Thus, this modified algorithm still guarantees the minimum expected traversal cost to the goal.

Note: One may be wondering why we did not simply run Breadth First Search from the initial node to determine the time attribute of all other squares. This is because while BFS computes the shortest number of moves to a particular square, this may not necessarily constitute the number of steps to the node along the optimal path. For example, if a node is 10 moves away, but the path is through high-cost terrain, another route which arrives after 12 steps may have a lower expected cost. The algorithm must use the correct number in calculation.

8.3 Dynamic Recalculation

Once the shortest expected-cost path is computed, the character takes one step along it. After this, the rain updates according to its Markov process across the board. At this point, we discard the previously computed path and re-run the algorithm from the character's new position with updated expected values. This process repeats until the character reaches the goal. While this seems redundant and expensive, it ensures correctness throughout the game, as we are always acting according to the newest information.

9 Pseudocode

Algorithm 1 2×2 Matrix Multiplication

```
function MatrixMult(A, B)                    ▷ A and B are 2 × 2 matrices.
    C ← Empty 2D Array
    for i = 1 to i = 2 do
        for j = 1 to j = 2 do
            C[i][j] = A[i][1] · B[1][j] + A[i][2] · B[2][j]
        end for
    end for
    return C
end function
```

Algorithm 2 Matrix Powers

```
function Power(M, n)
    if n = 0 then
        I = [[1, 0], [0, 1]]
        return I
    else
        M^(n−1) = Power(M, n − 1)
        return MatrixMult(M, M^(n−1))
    end if
end function
```

These are preliminary algorithms that provide the foundation for the calculation of expected costs involving matrices. The first algorithm encodes the process of 2×2 matrix multiplication in terms of multi-dimensional arrays (a primitive data type). The second algorithm recursively defines matrix powers by decreasing the exponent at each step. The base case is M^0 which is simply the identity.

Algorithm 3 Terrain Dependent Cost Value

```
function TerrainCost(u)
    a ← 0
    if TerrainType(u) = Field then
        a ← 0.5
    else if TerrainType(u) = Forest then
        a ← 1
    else
        a ← ∞
    end if
    return a
end function
```

Algorithm 4 Expected Cost Between Two Nodes

```
function ExpectedCost(u, v, P)
    n ← Steps(u)
    a ← TerrainCost(u)
    b ← TerrainCost(v)
    P^n ← Power(P, n)
    if IsRain(u) = True then
        c_1 ← 2a · P^n[1][1] + a · P^n[1][2]
    else
        c_1 ← 2a · P^n[2][1] + a · P^n[2][2]
    end if
    if IsRain(v) = True then
        c_2 ← 2b · P^n[1][1] + b · P^n[1][2]
    else
        c_2 ← 2b · P^n[2][1] + b · P^n[2][2]
    end if
    return c_1 + c_2
end function
```

Algorithm 5 Next Square Along Optimal Path

```
1: function TRACE(Start, Goal, Map: G)
2:     Current ← Goal
3:     while PREDECESSOR(Current) ≠ Start do
4:         Current ← PREDECESSOR(Current)
5:     end while
6:     return Current
7: end function
```

Algorithm 6 Dijkstra's Algorithm with Expected Costs

```
1: function EXPECTEDDIJKSTRA(Start, Goal, Map: G, P)
2:     for all u ∈ G do
3:         DIST(u) ← ∞
4:         STEPS(u) ← null
5:         PREDECESSOR(u) ← null
6:     end for
7:     DIST(Start) ← 0, STEPS(Start) ← 0
8:     PQ ← Empty minimum ranking priority queue
9:     Enqueue Start to PQ with priority 0
10:    while PQ is not empty do
11:        x ← PQ.peek()
12:        PQ.dequeue()
13:        if x = Goal then
14:            if DIST(Goal) < ∞ then
15:                return TRACE(Start, Goal, Map: G)
16:            else
17:                return 'No Finite Path Found'
18:            end if
19:        end if
20:        for all v ∈ NEIGHBOURS(x) do
21:            newDist ← DIST(x) + EXPECTEDCOST(x, v, P)
22:            if newDist < DIST(v) then
23:                DIST(v) ← newDist
24:                STEPS(v) ← STEPS(u) +1
25:                PREDECESSOR(v) ← u
26:                PQ.enqueue(v) with priority DIST(v)
27:            end if
28:        end for
29:    end while
30: end function
```

Algorithm 7 Dynamic Character Movement

```
function MoveCharacter(Start, Goal, Map: G, P)
    Position ← Start
    while Position ≠ goal do
        Next ← ExpectedDijkstra(Position, Goal, Map: G, P)
        Move character to node Next
        Position ← Next
        Map ← Updated Map
    end while
    return Goal Reached
end function
```

10 Algorithm Execution Example

To illustrate how the algorithm functions, we apply it to a sample 2×3 map (Figure 4), where the character attempts to move from the bottom left square to the top right. Underneath we have labelled the nodes $A - F$ for reference in the calculation.

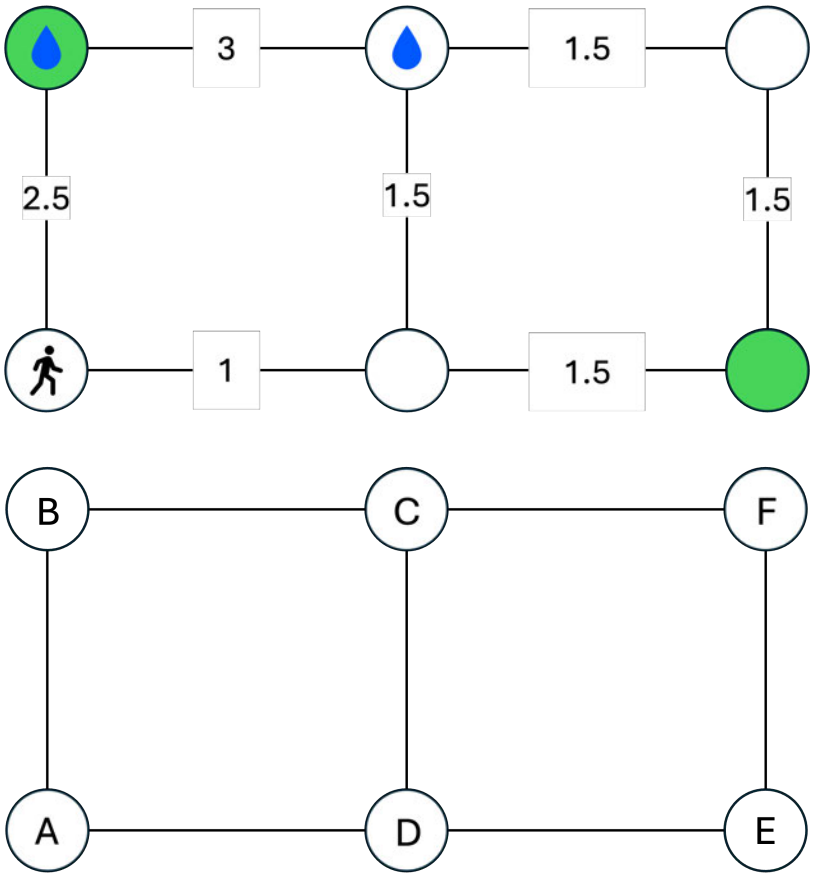




Figure 4: Top: Example 2×3 board state. Bottom: Labelling of the nodes $A - F$.

10.1 Calculation

Vertex	A	B	C	D	E	F
–	0	∞	∞	∞	∞	∞
A	0	2.5_A	∞	1_A	∞	∞
D	0	2.5_A	2.4_D	1_A	2.65_D	∞
C	0	2.5_A	2.4_D	1_A	2.65_D	3.74_C
B	0	2.5_A	2.4_D	1_A	2.65_D	3.74_C
E	0	2.5_A	2.4_D	1_A	2.65_D	3.74_C
F	0	2.5_A	2.4_D	1_A	2.65_D	3.74_C

The table shows how the priority queue is dequeued, and how the shortest expected path estimates evolve as the algorithm progresses. Subscripts indicate the assigned predecessor for each node during the computation. Although each node also maintains a 'Steps' property — used to calculate expected costs once dequeued — we have omitted it here for clarity. Based on the current predecessor assignments, the fastest expected path is $A \to D \to C \to F$ with a total expected cost of 3.74 time units (although the algorithm only finds the next move). Thus, the character will move to D (one square to the right), and the algorithm will be rerun after the map updates. For simplicity, we present only a single iteration; future iterations follow a similar structure, beginning from the character's new position.

11 Evaluation and Justification

11.1 Model Creation

Storing the map of Griddo as a weighted, undirected graph was the most effective and natural way to model the problem. This abstraction allows us to apply well-known graph algorithms such as Dijkstra's algorithm while preserving the important aspects of the real-world environment the game simulates. Each square (tile) on the board becomes a node, and possible transitions between adjacent squares become edges. As the character can move back and forth across two squares, the graph is undirected. Additionally, the weights are designed to model the true time taken to cross between two squares by calculating time to cross half of each square. We do this as it may be a useful metric in the game, or required for other calculation such as a points system. This graph structure not only models possible movements but also captures the evolving difficulty of movement based on rain conditions.

There were no simpler or more logical alternatives to a graph-based model. While a raw grid or matrix representation could store terrain and rain status, it would not allow for elegant

implementation of path-finding algorithms or efficient updates of traversal costs.

Furthermore, we decided to store properties of squares such as their terrian type or rain status as node attributes. This was done instead of storing the information in a dictionary, array or list so we can directly query the nodes, simplifying implementation, and readability. Also, by storing things as node properties, the node is self-contained which is the primary goal of object-oriented programming.

We also chose to model the rain as a discrete time Markov chain, independent for each square. This sacrifices some realism as rain is not confined to squares, and does not necessarily evolve once a character has completed its move. Additionally, as each square is independent, over time we lose some of the clustering properties of rain, and how they are wind-driven in the real world. This trade was made in order to allow noticeable patterns in rain, rather than make it too complex, to the point it is practically random. It is also important to note that the model is being created for a video game, and thus complete realism is not entirely required, and could detract from the game. For example, rain without borders would make the game difficult to play optimally or challenging to understand.

Finally, the Expected Dijkstra'a Algorithm incorporated expected edge weights as opposed to the model's edge weights. We decided against storing these expected weights on the nodes/edges and instead computing them when necessary. This is because the expected edge weights in our context are dynamic — they depend not only on the states of neighbouring squares, but on the number of steps taken up till that point which cannot be predetermined. Additionally, after the map updates, several of these pre-computed expected costs would become redundant, as rain occupies different squares. Hence, keeping the graph structure separate from transient expected costs simplifies both the idea and the implementation. The graph preserves the static terrain (rainy/dry properties) while expected costs are computed dynamically during algorithm execution.

The real-world features we modeled include:

- The terrain type of each square (rain-slowed or dry),
- The stochastic evolution of rain conditions, modelled via a discrete-time Markov chain,
- The traversal cost at any time step, based on terrain and rain state.

These abstractions capture all the salient features necessary to simulate rational movement under uncertainty. Although some nuances of real weather patterns (like local clustering or wind-driven rain) are omitted, the model strikes a balance between realism and tractability. The abstraction is consistent across the model: all times are measured in 'time units', and rain transitions are governed by the same probability matrix P throughout.

11.2 Algorithm Design

The dynamic pathfinding algorithm developed in this report is a well-suited and coherent solution for navigating the Griddo terrain under the influence of stochastic weather (rain) conditions. It satisfies the key criteria of realism, adaptiveness, and computational viability.

To begin with, we decided to consider the expected values of paths as opposed to their initial cost. The use of expected edge weights provides a mathematically principled approach to decision-making under uncertainty as it avoids choosing locally optimal paths when weights will most likely ultimately change. Figure 3 gives an example of a board state where a path is deemed shorter ignoring board changes, but the expected cost actually turns out to be greater. Additionally, determining the best move from the weights in the initial board state alone can be seen as assuming that the same initial board state will continue over several time steps which will only occur with a very small probability. Instead, using the expected cost of a path (the average cost experienced) will give a better indication of the true cost faced by the character, because the costs are centered around the mean. This ensures that the algorithm remains context-aware and provides paths that are optimal given the best available forecast of future rain conditions.

Additionally, several alternative approaches to the problem of least expected cost paths were considered, including Bellman-Ford, Floyd-Warshall, and brute-force enumeration. Brute-force, while the simplest to implement and with a guarantee of optimality (checks every option), was discarded early because it is computationally infeasible. The number of possible paths grows factorially with the board size and returning the path efficiently was a criteria of the game. Additionally, while Floyd-Warshall is powerful for finding all-pairs shortest paths, it would introduce massive overhead and would be unnecessary since we only seek a path from a single start to a single goal. Additionally, as we seek to find the expected costs of paths and not their distance, the method used in Floyd-Warshall's algorithm would not work as it does not keep track of path length, which is needed to correctly calculate the expected cost of traversing an edge at a particular time. Finally, we did not use Bellman-Ford as the expected values were all positive and Bellman-Ford is slower than Dijkstra's algorithm.

Our modified use of Dijkstra's algorithm avoids enumerating all possible paths, instead adopting a greedy approach (on the expected values). Dijkstra's algorithm has the benefit that it is specifically designed for single-source shortest path problems halting upon reaching the goal node, avoiding redundant computation. Additionally, the algorithm is rather flexible, in that it can account for evolving rain probabilities by using expected value $\mathbb{E}[X_i \mid \mathcal{H}]$ of traversal time as a proxy for edge weights. With this change, and the correct relaxation step based on the steps taken so far, the algorithm builds the path incrementally based on expected cost, which is efficient in both space and time.

Finally, the algorithm avoids the pitfall of blindly committing to a fixed path upfront. This is particularly important in stochastic environments, where early forecasts may diverge significantly from actual events. The algorithm is thus able to compute and move the character along the best path, ensuring global optimality (as far as probability goes) under the model.

11.3 Limitations and Improvements

Despite its effectiveness, the model does have limitations. It assumes that the Markov model for rain is perfectly known and stationary, which may not always hold in practice. Additionally,

the algorithm recomputes the entire expected-cost path from the character's current position to the goal after every move. While this ensures the path is always optimal based on the current state, it is computationally expensive and often redundant as the path does not change much between moves. The total cost of our approach is proportional to the number of steps taken to reach the goal, multiplied by the cost of one algorithm run.

To counter the computational inefficiency, a natural improvement would be to incorporate a heuristic, resulting in an A*-like algorithm. For instance, one could use the Manhattan distance or an admissible approximation of expected cost as a heuristic. This would cause the algorithm to favour nodes closer to the goal, as opposed to sprawling outward from the start node. With the correct heuristic, the algorithm would still provide optimal paths while reducing the number of node relaxations.

Another compromise is to recompute the path not after every move but every k moves. The character would follow the planned path for k steps unless a significant change is detected (e.g., drastic rainfall within some radius), at which point the path can be recalculated early. As the path usually remains constant between moves, by taking multiple steps before reevaluating we reduce the number of times the algorithm is run, and result in a similar score.

11.4 Conclusion

In this report, we presented a context-aware approach to dynamic pathfinding in a stochastic environment, tailored specifically to the constraints and gameplay mechanics of Griddo. By using a graph-based model and representing rain evolution as a Markov process, we balanced realism and computational tractability. The use of Expected Dijkstra's Algorithm - relying expected edge weights — also allowed the character to avoid short sighted actions, and make more natural choices. Our approach enables a rational agent to navigate a partially predictable, evolving world, demonstrating how mathematical modelling and algorithmic thinking can be used to enhance decision-making in interactive systems.